

Sky Analytics Engine

Architettura distribuita per l'analisi in tempo reale del traffico aereo

Flavio Simonelli

Matricola 0365333

Dipartimento di Ingegneria

Università di Roma "Tor Vergata"

flavio.simonelli@alumni.uniroma2.eu

Francesco Masci

Matricola 0365258

Dipartimento di Ingegneria

Università di Roma "Tor Vergata"

francesco.masci.02@alumni.uniroma2.eu

Sommario—Questo lavoro presenta *Sky Analytics Engine*, una piattaforma distribuita per l'analisi in tempo reale di voli. Un simulatore sviluppato in Go ricostruisce l'*event time*, ordina gli eventi e ne esegue un replay accelerato verso Apache Kafka. Il sistema calcola le prestazioni orarie delle principali compagnie, le classifiche degli aeroporti con più ritardi severi e la distribuzione dei ritardi tramite DDSketch. Watermark, *allowed lateness* e side output rendono esplicita la gestione degli eventi fuori ordine. I risultati e la telemetria confluiscono in InfluxDB tramite Telegraf e sono consultabili con Grafana. L'architettura è eseguibile localmente con Docker Compose e su un cluster Amazon EC2; il checkpointing *exactly-once*, con stato RocksDB incrementale su S3 o HDFS, supporta il recupero dai guasti. La campagna sperimentale confronta parallelismo, disordine temporale e frequenza dei checkpoint.

Index Terms—Apache Flink, stream processing, Apache Kafka, event time, watermark, DDSketch, checkpointing,

I. INTRODUZIONE

L'analisi tempestiva del traffico aereo richiede di aggiornare indicatori e classifiche mentre gli eventi vengono prodotti. In questo contesto non è sufficiente calcolare correttamente un risultato: occorre anche stabilire quando esso possa essere considerato abbastanza completo da essere pubblicato. Ritardi di rete, partizioni inattive e arrivi fuori ordine rendono infatti non coincidenti il tempo associato a un volo e l'istante in cui il sistema lo riceve.

Il progetto affronta questo problema a partire da circa 2,2 milioni di record del Bureau of Transportation Statistics, relativi al periodo 1 gennaio–30 aprile 2025. Il dataset storico viene utilizzato per generare un flusso riproducibile, nel quale la distanza temporale tra gli eventi può essere compressa e il grado di disordine controllato. In questo modo è possibile sottoporre il sistema a condizioni di carico differenti e valutare separatamente gli effetti di velocità di ingresso, parallelismo e ritardo degli eventi.

L'obiettivo è progettare una pipeline distribuita capace di sostenere tre analisi con caratteristiche diverse: aggregazioni orarie sulle compagnie, classifiche degli aeroporti su più orizzonti temporali e stima della distribuzione dei ritardi. La soluzione deve limitare la quantità di stato mantenuto, gestire esplicitamente i record tardivi, rendere osservabili prestazioni e risultati e recuperare dal fallimento di un nodo senza compromettere la coerenza dell'elaborazione.

La relazione descrive dapprima l'architettura e il percorso seguito dagli eventi, quindi approfondisce l'implementazione

PLACEHOLDER SCHEMA ARCHITETTUALE

```

Go Replay → Kafka → Flink
               ↘ Q1  ↘ Q2  ↘ Q3
Kafka output → Telegraf → InfluxDB → Grafana
Checkpoint → S3/HDFS
  
```

Figura 1. Architettura logica di Sky Analytics Stream.

delle tre analisi e l'infrastruttura di deployment. Le sezioni finali presentano la metodologia sperimentale, la strategia di verifica e i criteri adottati per valutare prestazioni, completezza dei risultati e tolleranza ai guasti.

II. ARCHITETTURA DEL SISTEMA

A. Visione d'insieme

L'architettura, sintetizzata in Fig. 1, è organizzata in cinque livelli. Il simulatore costituisce il livello di produzione e pubblica record JSON sul topic Kafka `flights-stream`. //TODO: sei partizioni Kafka disaccoppia la velocità del replay da quella dei consumatori e costituisce anche il bus di uscita, con topic distinti per query e finestra.

Un unico job Flink consuma il topic di ingresso, deserializza i record, assegna timestamp e watermark e applica un pre-processing condiviso. Dal flusso pulito si diramano le tre query. Questa topologia evita tre letture indipendenti da Kafka e riutilizza le trasformazioni comuni, pur mantenendo operatori, UID e stato separati per ciascun ramo.

I risultati serializzati in JSON sono inviati ai topic di uscita. Telegraf li converte nel modello di InfluxDB, mentre il reporter InfluxDB di Flink esporta throughput, latenza, backpressure, utilizzo delle risorse e statistiche dei checkpoint in un bucket separato. Grafana espone dashboard per risultati analitici, prestazioni Flink e comportamento durante il ripristino.

B. Livello di ingestion

1) *Replay accelerato in Go*: L'ingestion e il replay degli eventi storici sono gestiti da un componente custom sviluppato in Go. La scelta è motivata dalla necessità di disporre di un produttore leggero, capace di sostenere i profili di carico previsti, fino a circa 9.150 eventi/s nello scenario

43.200×, senza sottrarre risorse al cluster Flink. Il motore è disaccoppiato dalle strategie concrete attraverso le interfacce `Loader`, `Scheduler`, `Waiter` e `Sink`, assemblate tramite un *builder*. Il simulatore implementa le seguenti funzionalità:

- **Lettura streaming e controllo d'integrità:** al primo avvio l'applicazione verifica il digest SHA-1 dell'archivio locale e, se necessario, scarica nuovamente il dataset dall'URL configurato. Il contenuto `tar.gz` viene attraversato direttamente tramite le librerie standard di Go, selezionando i CSV di interesse senza estrarli preventivamente su disco. Ogni riga valida è convertita in un `FlightRecord` tipizzato; i campi numerici mancanti restano nulli e le righe malformate vengono scartate e segnalate.
- **Cache binaria in formato GOB:** i record caricati sono ordinati cronologicamente rispetto all'*event time* e serializzati in una cache locale Go Object Binary. Il file è prodotto tramite scrittura temporanea e rinomina atomica, evitando cache parziali in caso di interruzione. Le esecuzioni successive caricano direttamente la sequenza già ordinata, eliminando i costi ripetuti di decompressione, parsing e sorting.
- **Ricostruzione dell'event time e replay accelerato:** per ciascun volo il timestamp logico è ricavato in UTC combinando `YEAR`, `MONTH`, `DAY_OF_MONTH` e `CRS_DEP_TIME`. Il motore conserva separatamente `EventTime` e `PublishAt`; in modalità ordinata i due coincidono. Applicando il fattore di accelerazione s , l'attesa tra due emissioni è calcolata come

$$\Delta t_{\text{reale}} = \frac{t_{p,i} - t_{p,i-1}}{s}. \quad (1)$$

Dal tempo teorico viene sottratto il costo già sostenuto dall'invio precedente, limitando la deriva del replay. Il simulatore registra periodicamente anche lo *speedup* effettivamente ottenuto.

- **Attesa ibrida ad alta precisione:** con fattori di accelerazione elevati, gli intervalli tra record possono scendere sotto la granularità affidabile della sola *sleep*. Il `HybridWaiter` rilascia quindi la CPU durante la parte principale dell'attesa e usa uno *spin* soltanto nella coda finale, la cui soglia è configurabile e vale 2 ms per default. Entrambe le fasi sono cancellabili tramite il contesto Go, permettendo una terminazione ordinata su `SIGINT` e `SIGTERM`.
- **Generazione di disordine:** uno scheduler decoratore ritarda ciascun record con probabilità $P \in [0,1]$ di un valore casuale compreso tra 1 e δ_{max} minuti logici. Viene modificato esclusivamente `PublishAt`, lasciando inalterato l'*event time*. Il successivo ordinamento stabile per tempo di pubblicazione produce così uno stream realmente fuori ordine dal punto di vista di Flink, adatto a valutare watermark e *allowed lateness*.
- **Pubblicazione asincrona su Kafka:** i record sono serializzati in JSON mantenendo i nomi delle colonne BTS e inviati al topic `flights-stream` mediante

`kafka-go`. Il writer usa bilanciamento round-robin, batch fino a 100 messaggi o 100 kB, timeout di 2 ms e modalità asincrona, evitando che la conferma di ogni singolo messaggio alteri la temporizzazione del replay. In alternativa, un sink testuale consente test locali senza Kafka.

2) *Watermark e record tardivi:* La sorgente usa una strategia di watermark a disordine limitato; il ritardo del watermark, l'inattività delle partizioni e l'*allowed lateness* sono parametri indipendenti. Una partizione inattiva viene esclusa dal minimo globale dopo due minuti logici configurabili, evitando che blocchi indefinitamente l'avanzamento del watermark.

Ogni finestra conserva i record arrivati entro l'*allowed lateness*; quelli oltre il limite sono instradati in un side output. Un operatore dedicato ne conta quantità e ritardo, rendendo possibile confrontare due strategie: watermark conservativo, che aumenta completezza e latenza, e watermark aggressivo con finestra mantenuta aperta, che produce prima un risultato e lo aggiorna all'arrivo di dati ammessi.

3) *Qualità del dato:* Il pre-processing verifica i campi necessari e scarta le inconsistenze logiche non recuperabili. La semantica richiesta è applicata vicino a ciascuna query: Q1 conserva cancellazioni e deviazioni per calcolarne i tassi, mentre Q2 e Q3 escludono anticipatamente tali voli quando le statistiche richiedono soltanto voli completati. I ritardi negativi sono valori validi, corrispondenti a partenze anticipate, e non vengono azzerati. I campi numerici mancanti non contribuiscono alle aggregazioni che li richiedono.

C. Livello di processing

Apache Flink costituisce il livello computazionale. Il job costruisce una sola sorgente Kafka e un ramo condiviso di pre-processing, dal quale derivano le tre analisi. Gli operatori sono identificati mediante UID stabili, indispensabili per associare correttamente lo stato durante ripristino e aggiornamento del job. Filtri e proiezioni sono anticipati rispetto agli operatori keyed, riducendo serializzazione e traffico di rete; le aggregazioni sono incrementali e mantengono esclusivamente lo stato necessario.

D. Livello di persistenza

Kafka è impiegato anche come buffer dei risultati: ogni combinazione query-finestra dispone di un topic dedicato. Questa separazione consente a consumatori e politiche di retention di evolvere indipendentemente dalla topologia Flink. Telegraf traduce i messaggi JSON e li persiste in InfluxDB; i CSV richiesti dalla traccia sono prodotti come esportazione finale, preservando schema e ordine delle colonne.

E. Metrics store e visualizzazione

La telemetria applicativa è mantenuta in un bucket InfluxDB distinto dai risultati analitici. Il reporter di Flink esporta throughput, latenza, backpressure, stato dei checkpoint e utilizzo delle risorse, mentre gli operatori custom aggiungono misure sui record tardivi. Grafana interroga entrambi i bucket e permette di correlare l'andamento delle query con il comportamento del cluster.

III. IMPLEMENTAZIONI DELLE ANALISI DISTRIBUITE

A. Q1: prestazioni delle compagnie

Q1 filtra AA, DL, UA e WN, proietta i soli attributi necessari e partiziona per compagnia. Su finestre tumbling di un'ora, un'AggregateFunction aggiorna incrementalmente numero totale, completati, cancellati, deviati, somma e conteggio dei ritardi validi e partenze con ritardo superiore a 15 minuti. Il ProcessWindowFunction aggiunge gli estremi temporali e calcola media e percentuali. La memoria è $O(1)$ per coppia compagnia-finestra, poiché non vengono materializzati i record.

B. Q2: classifica degli aeroporti

Q2 considera i voli completati e raggruppa per aeroporto di origine su finestre tumbling di una e sei ore. Per ogni aeroporto l'accumulatore mantiene conteggio, somma, massimo, numero di ritardi superiori a 30 minuti e soltanto i 20 voli peggiori, ordinati per ritardo. Ciò limita lo stato pur producendo la lista richiesta.

Alla chiusura, i risultati locali sono raggruppati per estremo della finestra. Un processore keyed elimina gli aeroporti con meno di 30 voli, ordina per numero di ritardi severi e produce la Top-10. L'orizzonte "dall'inizio" è implementato con una GlobalWindow e un trigger continuo ogni ora di event time: lo stato non viene eliminato e ogni emissione rappresenta la classifica cumulativa aggiornata.

C. Q3: distribuzione dei ritardi

Q3 filtra le quattro compagnie e i voli completati, quindi partiziona per coppia (compagnia, fascia oraria programmata). Le finestre tumbling sono di uno e sette giorni; una finestra globale emette aggiornamenti giornalieri.

Memorizzare e ordinare tutti i ritardi avrebbe costo lineare nello stato. L'accumulatore usa invece DDSketch [2], una struttura unificabile che associa i valori a indici logaritmici. Con accuratezza relativa $\alpha = 0,01$, la stima $\hat{x}$ di un quantile positivo x soddisfa approssimativamente

$$|\hat{x} - x| \leq \alpha |x|. \quad (2)$$

La configurazione adotta una mappatura interpolata cubicamente e store densi limitati a 2048 bin; in presenza di valori estremi, i bin inferiori vengono collassati per mantenere memoria limitata e maggiore risoluzione sui ritardi elevati. Minimo e massimo sono conservati esattamente. Una serializzazione binaria personalizzata salva bin positivi, negativi e zeri, rendendo lo sketch compatibile con checkpoint e redistribuzione dello stato.

IV. INFRASTRUTTURA DI DEPLOYMENT

A. Ambiente locale containerizzato

Per lo sviluppo locale, Docker Compose avvia Kafka in modalità KRaft, un JobManager, tre TaskManager con due slot ciascuno, InfluxDB, Telegraf, Grafana e, quando necessario, HDFS. Alias DNS interni rendono la configurazione indipendente dagli indirizzi dei container.

Tabella I
PROFILI DI REPLAY PIANIFICATI.

Fattore	Durata	Throughput medio	1 h logica
1 440×	~ 120 min	~ 305 ev/s	2,5 s
14 400×	~ 12 min	~ 3 050 ev/s	0,25 s
43 200×	~ 4 min	~ 9 150 ev/s	0,083 s

B. Cluster distribuito su AWS EC2

Il deployment cloud usa template AWS CloudFormation per VPC e nodi EC2 e script distinti per broker, JobManager, TaskManager e stack di osservabilità. I container operano in `network_mode: host`; nomi privati stabili separano la configurazione dagli IP. Il job è distribuito come fat JAR Java 21 e lo scheduler adattivo di Flink assegna le risorse disponibili fino al parallelismo massimo configurato.

C. Tolleranza ai guasti

Il checkpointing è configurabile e usa semantica *exactly-once*, un solo checkpoint concorrente, pausa minima, timeout e numero di fallimenti tollerati. Lo stato è gestito da RocksDB con snapshot incrementali. In locale i checkpoint possono essere salvati su HDFS; su EC2 sono persistiti in S3 tramite IAM Instance Profile, così la perdita di un TaskManager non implica la perdita dello stato. La configurazione corrente usa intervallo di 5 minuti, pausa minima di 30 secondi, timeout di 3 minuti e checkpoint non allineati; tali valori saranno variati negli esperimenti.

V. VALUTAZIONE SPERIMENTALE

A. Metodologia e configurazione

A velocità reale il carico medio è circa

$$\frac{2,2 \cdot 10^6}{120 \cdot 24 \cdot 3600} \simeq 0,21 \text{ eventi/s}, \quad (3)$$

troppo basso per caratterizzare il sistema. Sono quindi previsti i profili in Tab. I. Ogni prova parte da topic e bucket puliti e usa la stessa versione del dataset. Le metriche principali sono throughput di sorgente e sink, latenza end-to-end, backpressure, utilizzo CPU/rete, durata e dimensione dei checkpoint e numero di record tardivi.

B. Analisi comparativa delle performance

Il primo esperimento confronta parallelismo 1, 3 e 6 ai fattori 14 400× e 43 200×, senza disordine aggiuntivo né checkpoint, per individuare il punto oltre il quale coordinamento e serializzazione annullano il beneficio dei worker.

Il secondo introduce fino a 30 minuti logici di disordine a 14 400×. Sono confrontati: (i) watermark di 30 minuti e lateness nulla; (ii) watermark di 5 minuti e lateness di 25 minuti; (iii) watermark di 10 minuti e lateness nulla. Oltre alla latenza saranno misurati completezza e aggiornamenti delle finestre.

Il terzo esperimento usa il miglior parallelismo individuato e confronta checkpoint ogni 5 minuti e ogni 30 secondi. Una prova aggiuntiva termina un TaskManager al sesto minuto per

misurare tempo di ripristino, perdita o duplicazione dei risultati e variazione del throughput.

DA COMPLETARE

Inserire grafico: throughput e latenza al variare di fattore di accelerazione e parallelismo.

DA COMPLETARE

Inserire grafico/tabella: completezza, late records e latenza per le tre strategie di watermark.

DA COMPLETARE

Inserire grafico: durata/dimensione checkpoint e recovery timeline con guasto del TaskManager.

C. Risultati preliminari

La pipeline e la raccolta automatica delle metriche sono operative, ma la campagna non è ancora conclusa. Per evitare di anticipare conclusioni non supportate, i valori finali e la relativa discussione saranno inseriti nei riquadri precedenti. In particolare dovranno essere riportati media, mediana e percentili della latenza, intervalli di osservazione stabili e configurazione delle istanze EC2. Sarà inoltre verificato che i CSV in `Results` rispettino schema e numerosità attese.

D. Discussione dei risultati analitici

1) *Contratto dei risultati*: I topic di uscita sono separati per evitare che finestre con frequenze e schemi differenti interferiscano tra loro. Q1 usa un topic; Q2 e Q3 ne usano tre ciascuna, rispettivamente per gli orizzonti 1h/6h/globale e 1d/7d/globale. I messaggi includono sempre il timestamp logico della finestra e le chiavi dimensionali. Per Q2 la chiave Kafka è l'identificativo dell'aeroporto di origine, così gli aggiornamenti dello stesso aeroporto conservano l'ordine all'interno di una partizione.

Telegraf consuma i sette topic, estrae il JSON e scrive in InfluxDB usando come tag soltanto le dimensioni a cardinalità controllata, quali compagnia, tipo di finestra e fascia oraria. Conteggi, ritardi e percentili rimangono campi numerici. Questa distinzione evita di trasformare misure ad alta variabilità in tag, che farebbero crescere eccessivamente la cardinalità dell'indice. Le dashboard Grafana interrogano lo stesso archivio sia per mostrare l'evoluzione delle query sia per correlare un'anomalia del risultato con backpressure, latenza o checkpoint.

La specifica richiede la consegna in CSV, mentre il percorso operativo usa JSON e InfluxDB. I CSV finali sono quindi artefatti di esportazione: per ogni query vengono mantenuti ordine e nomi delle colonne previsti dalla traccia, senza serializzare nel formato tabellare le metriche tecniche. Per Q2 la lista dei voli ritardati è rappresentata come un singolo campo quotato, in modo che le virgole interne non alterino il numero di colonne.

2) *Strategia di verifica*: La correttezza è controllata a più livelli. I test unitari verificano deserializzazione, aggregatori e processor di finestra su insiemi ridotti con risultato noto. I

test di integrazione avviano la topologia con sorgenti limitate e confrontano i record emessi con un calcolo di riferimento. Sono verificati esplicitamente i casi limite: volo cancellato o deviato, ritardo negativo, fascia oraria a mezzanotte, aeroporto sotto la soglia di 30 voli, parità nel ranking e valore estremo inserito in DDSketch.

Per le finestre è necessario distinguere correttezza del calcolo e completezza dell'input. A parità di insieme di record, l'aggregazione deve essere deterministica; modificando watermark e lateness, invece, può cambiare l'insieme arrivato prima della rimozione dello stato. Per questo i side output non sono trattati come semplici log: il loro conteggio entra nei risultati sperimentali e consente di spiegare eventuali differenze rispetto alla baseline ordinata.

La verifica della tolleranza ai guasti controlla che, dopo la terminazione forzata di un TaskManager, il job riparta dall'ultimo checkpoint completato e torni a elaborare allo stesso throughput. Oltre allo stato Flink va considerato il confine con Kafka: la semantica *exactly-once* del checkpoint protegge lo stato e le posizioni della sorgente; la validazione finale dovrà confermare anche l'assenza di duplicati osservabili nei topic e nei CSV, oppure documentare l'eventuale necessità di deduplicazione per chiave e timestamp.

DA COMPLETARE

Inserire tabella di validazione: test eseguiti, record attesi/osservati, errore DDSketch e verifica dei CSV.

VI. CONCLUSIONI E SVILUPPI FUTURI

Sky Analytics Stream realizza una pipeline distribuita basata sull'*event time*, progettata per elaborare in modo continuo un replay accelerato dei dati di volo. Le aggregazioni incrementali limitano lo stato delle tre query; DDSketch rende praticabile il calcolo online dei percentili; watermark, lateness e side output permettono di quantificare il compromesso tra tempestività e completezza. La separazione dei servizi facilita il deployment e l'osservabilità, mentre Kafka disaccoppia la produzione dal consumo degli eventi e Flink fornisce elaborazione con stato e meccanismi di recupero dai guasti.

La conclusione quantitativa dipenderà dagli esperimenti ancora in corso. Il lavoro residuo consiste nel completare i benchmark, inserire grafici e tabelle, validare i CSV finali e motivare la configurazione scelta per l'esecuzione dimostrativa.

RIFERIMENTI BIBLIOGRAFICI

- [1] Apache Software Foundation, "Apache Flink Documentation," [Online]. Disponibile: <https://flink.apache.org/>.
- [2] C. Masson, J. E. Rim, and H. K. Lee, "DDSketch: A fast and fully-mergeable quantile sketch with relative-error guarantees," *Proc. VLDB Endowment*, vol. 12, no. 12, pp. 2195–2205, 2019.
- [3] L. Fernando, H. Bindra, and K. Daudjee, "An experimental analysis of quantile sketches over data streams," in *Proc. EDBT*, 2023.

APPENDICE A
DICHIARAZIONE SULL'USO DI IA GENERATIVA

DA COMPLETARE

Completare e revisionare prima della consegna. Indicare strumenti utilizzati, attività supportate, parti significativamente generate o modificate e verifiche svolte dagli studenti. Per questa bozza, Codex (OpenAI) è stato impiegato per analisi della repository e supporto alla stesura; contenuti tecnici e risultati dovranno essere controllati dagli autori.